LAPORAN UJIAN TENGAH SEMESTER

Sistem Tertanam

"Monitoring Ruangan Sederhana Menggunakan Raspberry Pi 5 dengan Sensor DHT11, PIR, dan Buzzer"

Dosen Pengampu:
Drs. Hari Suparwito, S.J., M.App.IT

Disusun Oleh:

Filipus Ferry Pradana / 235314039
I Dewa Gede Khrisnayana Sathya Gautama / 235314114

PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS SANATA DHARMA

2026

1 Pendahuluan

1.1 Latar Belakang

Kondisi lingkungan dan keamanan fisik di dalam ruangan khusus, seperti ruang laboratorium atau ruang server, memegang peranan krusial dalam menjaga keandalan serta usia pakai perangkat elektronik di dalamnya. Suhu ruangan yang terlalu tinggi atau tingkat kelembaban udara yang ekstrem dapat mempercepat degradasi komponen perangkat keras, memicu korsleting akibat kondensasi, dan menurunkan performa sistem. Di samping faktor termal dan kelembaban, pemantauan aktivitas atau pergerakan manusia di dalam ruangan juga diperlukan sebagai lapisan keamanan fisik untuk mendeteksi keberadaan personel maupun potensi akses tidak sah. Oleh karena itu, diperlukan sebuah sistem berbasis *embedded system* yang mampu memantau parameter suhu, kelembaban udara, serta keberadaan gerakan secara *real-time* menggunakan sensor DHT11 dan sensor PIR (*Passive Infrared*), sekaligus memberikan peringatan bertingkat melalui indikator visual (LED) dan audio (*buzzer*).

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah pada proyek ini adalah:

1. Bagaimana merancang dan mengintegrasikan sensor suhu dan kelembaban (DHT11), sensor deteksi gerakan (PIR), tombol interupsi (*push button*), serta indikator output dengan Raspberry Pi?

2. Bagaimana mengimplementasikan logika *Finite State Machine* (FSM) untuk mengelola masa kalibrasi (*warm-up*) sensor PIR serta menentukan prioritas peringatan berdasarkan ambang batas suhu, kelembaban, dan deteksi gerakan?

3. Bagaimana kinerja dan keandalan sistem dalam merespons perubahan kondisi lingkungan, deteksi aktivitas gerakan, serta penekanan tombol peredam alarm (*mute*)?

1.3 Tujuan Proyek

Tujuan dari pelaksanaan proyek ini adalah:

1. Membangun purwarupa sistem monitoring ruangan sederhana berbasis Raspberry Pi yang mengintegrasikan pemantauan kondisi termal/kelembaban dan deteksi gerakan.

2. Mengimplementasikan pembacaan data sensor DHT11 dan sensor PIR secara berkala menggunakan pemrograman Python serta mengatur transisi indikator output (LED *Traffic Light* dan *Buzzer*) secara akurat.

3. Menguji responsivitas sistem terhadap berbagai skenario uji, meliputi masa *warm-up* sensor PIR, kondisi normal, peringatan waspada (kelembaban meningkat atau terdeteksi gerakan), kondisi kritis, hingga fitur peredaman suara alarm (*mute*).

2 Tinjauan Pustaka

2.1 Raspberry Pi

Raspberry Pi adalah komputer papan tunggal (*Single Board Computer* / SBC) seukuran kartu kredit yang dilengkapi dengan prosesor berbasis arsitektur ARM, memori RAM, port komunikasi, serta deretan pin GPIO. Raspberry Pi umumnya menjalankan sistem operasi berbasis Linux (seperti Raspberry Pi OS) dan mendukung pemrograman tingkat tinggi menggunakan bahasa Python untuk mengendalikan perangkat keras eksternal secara efisien.

2.2 Sensor Suhu dan Kelembaban (DHT11)

Sensor DHT11 merupakan modul sensor digital yang berfungsi mengukur suhu dan kelembaban relatif udara di sekitarnya. Sensor ini menggunakan elemen sensor kapasitif untuk mengukur kelembaban dan termistor (*Negative Temperature Coefficient*) untuk mengukur suhu, kemudian mengirimkan sinyal digital terkalibrasi melalui antarmuka komunikasi satu jalur (*single-wire serial interface*) ke pin GPIO Raspberry Pi.

2.3 Sensor Gerak PIR (*Passive Infrared*)

Sensor PIR (*Passive Infrared*) adalah sensor elektronik yang berfungsi mendeteksi pergerakan objek berdasarkan pancaran radiasi sinar inframerah pasif (panas tubuh) dari manusia atau benda hidup di sekitarnya. Istilah *passive* menunjukkan bahwa sensor ini tidak memancarkan sinyal inframerah sendiri, melainkan hanya menerima radiasi termal melalui lensa Fresnel menuju elemen sensor piroelektrik di dalamnya. Ketika terjadi perubahan pola radiasi inframerah akibat gerakan, sensor PIR akan mengubah logika pin keluaran digitalnya menjadi *HIGH* (3.3V). Saat pertama kali dialiri daya, elemen piroelektrik pada sensor PIR memerlukan waktu stabilisasi atau kalibrasi termal (*warm-up time*) sekitar 30 hingga 60 detik terhadap suhu latar ruangan agar dapat mendeteksi gerakan secara akurat tanpa memicu pembacaan palsu (*false trigger*).

2.4 Konsep Embedded System

Sistem tertanam (*embedded system*) adalah sistem komputasi khusus yang menggabungkan perangkat keras (*hardware*) dan perangkat lunak (*software*) yang dirancang untuk menjalankan fungsi pengawasan atau pengendalian spesifik. Sistem ini bekerja secara berkesinambungan dengan siklus utama menerima masukan dari berbagai sensor (*input*), memproses data sesuai algoritma atau *state machine* yang ditanamkan (*process*), dan menghasilkan aksi pada aktuator atau indikator (*output*).

2.5 Konsep Input-Output GPIO

GPIO (*General Purpose Input/Output*) adalah pin antarmuka fisik pada Raspberry Pi yang dapat dikonfigurasi secara fleksibel melalui perangkat lunak sebagai jalur masukan (*input*) maupun keluaran (*output*) sinyal digital. Sebagai *input*, pin GPIO membaca level logika tegangan tinggi (*HIGH* / 3.3V) atau rendah (*LOW* / 0V), seperti sinyal deteksi dari sensor PIR, untai data serial dari DHT11, maupun penekanan *push button*. Sebagai *output*, pin GPIO mengirimkan sinyal tegangan untuk mengaktifkan komponen indikator seperti modul LED dan *buzzer*.

3 Perancangan Sistem

3.1 Arsitektur Sistem

Secara garis besar, sistem monitoring ruangan sederhana ini terbagi menjadi tiga bagian utama: bagian masukan (*Input*), bagian pemrosesan (*Process*), dan bagian keluaran (*Output*). Diagram blok arsitektur sistem ditunjukkan pada skema di bawah ini:

DIAGRAM BLOK ARSITEKTUR SISTEM

[INPUT] Sensor Suhu & Kelembaban (DHT11) + Sensor Gerak (PIR) + Push Button

↓

[PROSES] Raspberry Pi (Akuisisi Data & Evaluasi Ambang Batas)

↓

[OUTPUT] Indikator LED / Buzzer + Tampilan Terminal / Layar

3.2 Desain Software dan Hardware

3.2.1 Skema Rangkaian

Berikut adalah representasi skema perancangan rangkaian perangkat keras yang menghubungkan sensor, *push button*, dan indikator ke Raspberry Pi:

[Tempatkan penjelasan detail rangkaian atau pengganti gambar skema rangkaian di sini]

(Catatan: Sensor DHT dihubungkan ke VCC 3.3V, GND, dan Pin Data GPIO; Push Button dihubungkan ke Pin GPIO dengan konfigurasi Pull-Up/Pull-Down; LED dihubungkan seri dengan resistor 220 Ω menuju Pin GPIO dan GND).

3.2.2 Penjelasan Pin GPIO

Pemetaan alokasi pin GPIO pada Raspberry Pi (menggunakan penomoran BCM) yang digunakan dalam sistem monitoring ruang lab/server ini disajikan pada Tabel 1.

Tabel 1: Pemetaan Pin GPIO Raspberry Pi

Komponen	**Pin BCM (GPIO)**	**Library / Objek**	**Fungsi / Mode**
Sensor DHT11 (Data)	GPIO 4 (`board.D4`)	`adafruit_dht.DHT11`	Input Data Suhu & Lembab
Sensor PIR	GPIO 23	`MotionSensor`	Input Deteksi Gerakan
Push Button (Mute)	GPIO 5	`Button` (Pull-Up)	Input Interupsi Mute Alarm
LED Merah	GPIO 17	`TrafficLights (red)`	Output Indikator Kritis
LED Kuning	GPIO 27	`TrafficLights (yellow)`	Output Indikator Waspada
LED Hijau	GPIO 22	`TrafficLights (green)`	Output Indikator Normal/Init
Active Buzzer	GPIO 25	`Buzzer`	Output Alarm Audio

3.2.3 State Machine

Sistem dirancang menggunakan pendekatan *Finite State Machine* (FSM) dengan lima kondisi (*state*) utama yang diatur berdasarkan skala prioritas kedaruratan seperti pada Tabel 3.

Tabel 2: Transisi State Machine Sistem Monitoring Ruangan

State	Kondisi Pemicu (Trigger)	Prioritas	Aksi Output (LED & Buzzer)
`INITIALIZING`	Waktu *warm-up* PIR < 60 detik & Suhu/RH aman	Awal	LED Hijau ON, Buzzer OFF
`NORMAL`	Suhu $< 31°C$, RH $< 75\%$, Tidak ada gerakan	Normal	LED Hijau ON, Buzzer OFF
`WARNING`	RH $\geq 75\%$ ($< 80\%$) ATAU terdeteksi gerakan PIR	Prioritas 2	LED Kuning ON, Buzzer Beep Lambat (0.2s ON / 0.8s OFF)
`CRITICAL`	Suhu $\geq 31°C$ ATAU RH $\geq 80\%$	Prioritas 1	LED Merah Kedip Cepat (0.2s), Buzzer Beep Cepat (0.1s)
`MUTED`	Tombol ditekan saat `WARNING` / `CRITICAL`	Override Audio	LED Kuning & Merah ON, Buzzer OFF

3.2.4 Flowchart Sistem

Alur kerja perangkat lunak pada sistem monitoring ruangan ini terdiri dari tahapan berurutan berikut:

1. **Mulai (Start):** Sistem dinyalakan dan program dijalankan.
2. **Inisialisasi:** Mengatur mode penomoran pin GPIO (BCM), mengatur arah pin (input/output), dan menginisialisasi objek sensor DHT.
3. **Baca Input:** Membaca status *push button* serta mengakuisisi nilai suhu (°C) dan kelembaban (%) dari sensor DHT.
4. **Evaluasi Kondisi:**
   - Jika tombol ditekan $\rightarrow$ Reset alarm / tampilkan status manual di layar.
   - Jika Suhu $>$ [Batas Suhu, misal 30°C] ATAU Kelembaban $>$ [Batas Kelembaban, misal 75%] $\rightarrow$ Masuk ke kondisi **Peringatan (Warning)**: Nyalakan LED Merah/Buzzer, matikan LED Hijau.
   - Jika Suhu dan Kelembaban berada di bawah batas $\rightarrow$ Masuk ke kondisi **Normal**: Nyalakan LED Hijau, matikan LED Merah/Buzzer.
5. **Tampilkan Data:** Mencetak nilai suhu, kelembaban, dan status ruangan ke terminal.
6. **Delay & Loop:** Tunggu selama 2 detik, lalu kembali ke langkah **Baca Input** hingga program dihentikan pengguna.

3.2.5 State Machine

Sistem dirancang menggunakan pendekatan *Finite State Machine* (FSM) dengan empat kondisi (*state*) utama seperti pada Tabel 3.

4 Implementasi

4.1 Tools & Library

Perangkat lunak dan pustaka (*library*) yang digunakan dalam tahap implementasi proyek ini meliputi:

Tabel 3: Transisi State Machine Sistem Monitoring Ruangan

State Saat Ini	Kondisi Pemicu (Trigger)	State Berikutnya	Aksi Output
`IDLE / INIT`	Inisialisasi selesai	`NORMAL`	Semua LED mati
`NORMAL`	Suhu $\leq 30^{\circ}$C & RH $\leq 75\%$	`NORMAL`	LED Hijau ON, Alarm OFF
`NORMAL`	Suhu $> 30^{\circ}$C atau RH $> 75\%$	`WARNING`	LED Hijau OFF, Alarm ON
`WARNING`	Push Button ditekan	`MUTE / ACK`	LED Merah berkedip, Buzzer OFF
`WARNING / MUTE`	Suhu $\leq 30^{\circ}$C & RH $\leq 75\%$	`NORMAL`	LED Hijau ON, Alarm OFF

- **Bahasa Pemrograman:** Python 3 pada lingkungan sistem operasi berbasis Linux (Raspberry Pi OS).

- **Library** `gpiozero`**:** Menggunakan kelas abstraksi perangkat keras tingkat tinggi meliputi `TrafficLights` (mengelola tiga indikator LED sekaligus), `Buzzer`, `Button` (dengan fitur *internal pull-up* dan *software debouncing*), serta `MotionSensor` untuk sensor PIR.

- **Library** `adafruit_dht` **&** `board`**:** Pustaka CircuitPython untuk mengakuisisi data digital dari sensor DHT11 melalui pin `board.D4` dengan konfigurasi `use_pulseio=False` agar stabil di Raspberry Pi.

- **Library** `time`**:** Menggunakan `time.monotonic()` untuk penghitungan waktu kalibrasi (*warm-up*) sensor PIR yang bebas dari gangguan perubahan jam sistem, serta `time.sleep()` untuk interval *polling* siklus utama.

4.2 Struktur Program

Program dibagi ke dalam beberapa blok fungsional yang modular, yaitu:

1. **Inisialisasi Hardware & Konfigurasi Sistem:** Mendefinisikan objek komponen beserta nomor pin GPIO (BCM) serta menetapkan konstanta ambang batas (`BATAS_SUHU_TINGGI = 31.0`$^{\circ}$C, `BATAS_LEMBAB_WASPADA = 75.0`%, `BATAS_LEMBAB_KRITIS = 80.0`%, dan `WAKTU_WARMUP_PIR = 60` detik).

2. **Handler Interupsi Tombol (**`tekan_tombol_handler`**):** Fungsi *callback* asinkron yang terhubung ke `tombol_mute.when_pressed`. Fungsi ini mengubah status `is_muted` secara *toggle* hanya ketika sistem sedang berada dalam kondisi `WARNING`, `CRITICAL`, atau `MUTED`.

3. **Manajemen Kalibrasi PIR (**`periksa_warmup_pir`**):** Menghitung mundur waktu stabilisasi elemen piroelektrik sensor PIR selama 60 detik pertama menggunakan `time.monotonic()` sebelum mengizinkan pembacaan deteksi gerakan.

4. **Akuisisi Sensor Tangguh (**`baca_semua_sensor`**):** Membaca nilai suhu dan kelembaban di dalam blok `try-except RuntimeError` untuk mencegah program berhenti paksa ketika terjadi kegagalan *checksum* pada DHT11, serta membaca status gerakan PIR jika masa *warm-up* telah selesai.

5. **Mesin Evaluasi State (**`evaluasi_state`**):** Menentukan transisi kondisi sistem berdasarkan hierarki prioritas (Prioritas 1 untuk `CRITICAL`, Prioritas 2 untuk `WARNING`, dan kembali ke `NORMAL` sekaligus me-reset status `is_muted` secara otomatis jika kondisi kembali aman).

6. **Pengendali Aktuator (`jalankan_output`):** Mengatur pola nyala/kedip LED dan frekuensi bunyi *buzzer* secara *non-blocking* di latar belakang (`background=True`) setiap kali terjadi perubahan *state*.

4.3 Cuplikan Kode

Berikut adalah cuplikan fungsi evaluasi *state* dan kontrol aktuator yang menjadi inti pengambilan keputusan pada sistem:

```
def evaluasi_state(suhu, kelembaban, gerakan):
    global current_state, is_muted

    # Prioritas 1: kondisi kritis
    if suhu >= BATAS_SUHU_TINGGI or kelembaban >=
       BATAS_LEMBAB_KRITIS:
        state_baru = "CRITICAL"
    # Prioritas 2: kondisi peringatan
    elif kelembaban >= BATAS_LEMBAB_WASPADA or gerakan:
        state_baru = "WARNING"
    else:
        state_baru = "NORMAL"
        is_muted = False

    # Mute tidak mengubah kondisi sensor
    if state_baru in ("WARNING", "CRITICAL") and is_muted:
        current_state = "MUTED"
    else:
        current_state = state_baru

    return current_state
```

Kode 1: Cuplikan Fungsi Evaluasi State dan Kontrol Output

5 Pengujian dan Hasil

5.1 Skenario Uji

Pengujian dilakukan melalui enam skenario yang mencakup masa kalibrasi awal, kondisi lingkungan normal, tingkat peringatan bertingkat (waspada hingga kritis), deteksi gerakan, serta fungsi peredaman suara (*mute*). Rekapitulasi skenario dan hasil uji ditunjukkan pada Tabel 4.

Tabel 4: Rekapitulasi Skenario dan Hasil Pengujian Sistem

No.	Skenario Uji	Kondisi Input / Parameter Uji	Respons Output Sistem	Status
1	Inisialisasi (*Warm-up*)	Waktu jalan < 60 dtk, Suhu $< 31°C$, RH $< 80\%$	State `INITIALIZING`, LED Hijau ON, PIR diabaikan	Berhasil
2	Kondisi Normal	Waktu ≥ 60 dtk, Suhu $< 31°C$, RH $< 75\%$, PIR Aman	State `NORMAL`, LED Hijau ON, Buzzer OFF	Berhasil
3	Kelembaban Waspada / Gerak	RH 75%–79% ATAU Sensor PIR mendeteksi gerakan	State `WARNING`, LED Kuning ON, Buzzer Beep Lambat	Berhasil
4	Suhu / Kelembaban Kritis	Suhu $\geq 31°C$ ATAU RH $\geq 80\%$ (termasuk saat *warm-up*)	State `CRITICAL`, LED Merah Kedip Cepat, Buzzer Cepat	Berhasil
5	Penekanan Tombol (Saat Alarm)	Push button ditekan saat state `WARNING` / `CRITICAL`	State `MUTED`, LED Kuning & Merah ON, Buzzer OFF	Berhasil
6	Penekanan Tombol (Saat Aman)	Push button ditekan saat state `NORMAL` / `INITIALIZING`	Pesan penolakan di terminal, state tidak berubah	Berhasil

5.2 Analisis dan Hasil Pengujian

Berdasarkan implementasi kode program dan skenario pengujian yang telah dijalankan, diperoleh analisis teknis sebagai berikut:

1. **Kesesuaian Logika Prioritas dan Penanganan Masa Kalibrasi (*Warm-up*):** Sensor PIR membutuhkan waktu stabilisasi termal sekitar 60 detik saat pertama kali dinyalakan agar tidak memicu alarm palsu (*false positive*). Kode program berhasil menangani hal ini melalui fungsi `periksa_warmup_pir()` yang menahan status sistem pada `INITIALIZING` (LED Hijau menyala) dan mengabaikan pembacaan PIR. Namun, program dilengkapi logika keselamatan cerdas: apabila selama masa *warm-up* terjadi kondisi darurat (suhu $\geq 31.0^{\circ}$C atau kelembaban $\geq 80.0\%$), sistem langsung memotong proses *warm-up* dan masuk ke state `CRITICAL`.

2. **Mekanisme Interupsi Tombol Mute dan Auto-Reset:** Penggunaan *event-driven callback* (`tombol_mute.when_pressed`) dengan parameter `bounce_time=0.1` detik membuat respons tombol sangat instan tanpa harus menunggu jeda `time.sleep(2.0)` pada siklus utama selesai. Ketika alarm diheningkan (`MUTED`), buzzer berhenti berbunyi tetapi LED Kuning dan Merah dinyalakan secara bersamaan sebagai penanda visual bahwa bahaya belum berakhir. Saat kondisi ruangan kembali ke `NORMAL`, variabel `is_muted` otomatis diatur ulang menjadi `False`, sehingga alarm siap berbunyi kembali jika terjadi bahaya berikutnya.

3. **Kendala yang Muncul dan Solusi Perangkat Lunak:**

   - **Kegagalan Baca Sensor DHT11 (*RuntimeError*):** Sensor DHT11 menggunakan protokol komunikasi satu jalur yang sangat sensitif terhadap waktu (*timing*), sehingga kerap memunculkan eksepsi `RuntimeError` (seperti kegagalan *checksum* atau *buffer* belum siap). Kendala ini berhasil diatasi dengan membungkus pembacaan sensor di dalam blok `try-except RuntimeError` serta menyimpan nilai pembacaan terakhir yang valid ke dalam variabel `suhu_valid` dan `lembab_valid` (dengan nilai awal *default* 27.0°C dan 60.0%).
   - **Transisi Output Tanpa Kedipan Berulang (*State Change Detection*):** Jika fungsi pengendali LED dan buzzer dipanggil ulang pada setiap iterasi *loop* (setiap 2 detik), pola kedipan (`blink`/`beep`) akan terus ter-reset dari awal. Penambahan variabel pelacak `status_output_terakhir` memastikan fungsi `jalankan_output()` hanya dieksekusi satu kali saat terjadi perpindahan *state*.

6 Pembahasan

6.1 Evaluasi Performa Sistem

Secara arsitektural, sistem bekerja dengan sangat efisien dan responsif. Pemanfaatan fitur `background=True` pada metode `blink()` dan `beep()` di pustaka `gpiozero` memungkinkan pembangkitan sinyal PWM/kedipan LED (siklus 0.2 detik) dan ketukan buzzer (siklus 0.1 detik pada kondisi kritis) berjalan pada *thread* terpisah. Hal ini membuat *main loop* tetap dapat berfokus pada pembacaan sensor setiap 2.0 detik tanpa memblokir frekuensi kedipan indikator. Selain itu, penggunaan `time.monotonic()` menjamin akurasi perhitungan waktu *warm-up* karena nilainya tidak terpengaruh apabila sistem operasi melakukan sinkronisasi waktu jaringan

(NTP). Pada saat program dihentikan (`KeyboardInterrupt`), blok `finally` juga memastikan seluruh objek perangkat keras ditutup (`close()` dan `dht_device.exit()`) sehingga tidak menyisakan pin GPIO yang menggantung.

6.2 Kelebihan dan Kekurangan

- **Kelebihan:**

  1. **Toleransi Kesalahan Tinggi (*Fault-Tolerant*):** Program tidak akan berhenti mendadak (*crash*) saat sensor DHT11 gagal merespons berkat mekanisme penyimpanan nilai valid terakhir (*last known good value*).
  2. **Indikasi Bertingkat yang Informatif:** Membedakan secara tegas antara peringatan dini (`WARNING` dengan LED kuning dan beep lambat) serta kondisi bahaya (`CRITICAL` dengan LED merah berkedip cepat dan beep cepat).
  3. **Interaksi Tombol Non-Blocking:** Penekanan tombol (*mute*) diproses secara asinkron menggunakan *event listener* dan dilengkapi proteksi *software debouncing* 100 ms.

- **Kekurangan:**

  1. **Keterbatasan Akurasi DHT11:** Sensor DHT11 memiliki resolusi pembacaan yang terbatas dan rentang *sampling rate* maksimal 1 Hz (1 kali per detik), sehingga kurang ideal untuk mendeteksi lonjakan suhu mikro secara instan.
  2. **Tidak Adanya Histeresis Ambang Batas:** Apabila suhu ruangan berfluktuasi tipis di sekitar angka ambang batas (misalnya naik-turun antara 30.9°C dan 31.0°C), sistem dapat berpindah-pindah *state* antara `NORMAL` dan `CRITICAL` secara berulang (*chattering*).
  3. **Penyimpanan Data Masih Volatil:** Log pembacaan sensor hanya ditampilkan ke layar terminal (*standard output*) dan belum disimpan ke dalam berkas log (*file CSV*) maupun basis data.

6.3 Potensi Pengembangan

Untuk meningkatkan keandalan sistem di lingkungan ruang server atau laboratorium yang sesungguhnya, beberapa pengembangan yang dapat dilakukan meliputi:

1. **Penambahan Logika Histeresis (*Deadband*):** Menerapkan batas bawah pemulihan yang berbeda dari batas atas pemicu (contoh: masuk state `CRITICAL` saat suhu ≥ 31.0°C, namun baru kembali ke `NORMAL` ketika suhu sudah turun hingga ≤ 29.5°C) untuk mencegah osilasi *state*.

2. **Pencatatan Data (*Data Logging*) & Integrasi IoT:** Menambahkan modul pencatatan otomatis ke basis data lokal (SQLite/InfluxDB) serta mengirimkan peringatan darurat secara *real-time* melalui bot Telegram atau protokol MQTT.

3. **Otomasi Aktuator Pendingin:** Menambahkan modul *relay* untuk menyalakan kipas *exhaust* atau unit AC cadangan secara otomatis ketika sistem memasuki kondisi `CRITICAL`.

7 Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa:

1. Sistem monitoring ruangan sederhana berbasis Raspberry Pi berhasil dirancang dan diimplementasikan dengan mengintegrasikan sensor suhu dan kelembaban (DHT), *push button*, serta indikator output LED/Buzzer melalui pin GPIO.

2. Perangkat lunak yang dibangun menggunakan bahasa Python mampu mengakuisisi data sensor secara berkala dan mengeksekusi transisi *state machine* secara akurat sesuai ambang batas yang ditentukan.

3. Berdasarkan empat skenario pengujian (kondisi normal, suhu tinggi, kelembaban tinggi, dan penekanan tombol), sistem mampu merespons perubahan kondisi lingkungan dan interaksi pengguna sesuai dengan tujuan proyek.

Lampiran

A. Kode Lengkap Program (`main.py`)

```
import time
import board
import adafruit_dht


from gpiozero import (
    TrafficLights,
    Buzzer,
    Button,
    MotionSensor,
)


# ============================================================
# 1. INISIALISASI HARDWARE
# ============================================================


lampu = TrafficLights(
    red=17,
    yellow=27,
    green=22,
)


buzzer = Buzzer(25)


tombol_mute = Button(
    5,
    pull_up=True,
    bounce_time=0.1,
)


sensor_pir = MotionSensor(23)


dht_device = adafruit_dht.DHT11(
    board.D4,
    use_pulseio=False,
)


# ============================================================
# 2. KONFIGURASI SISTEM
# ============================================================
```

```
48
49
50  WAKTU_WARMUP_PIR = 60  # detik
51
52
53  BATAS_SUHU_TINGGI = 31.0
54  BATAS_LEMBAB_WASPADA = 75.0
55  BATAS_LEMBAB_KRITIS = 80.0
56
57
58  current_state = "INITIALIZING"
59  is_muted = False
60  pir_siap = False
61
62
63  # ============================================================
64  # 3. TOMBOL MUTE
65  # ============================================================
66
67
68  def tekan_tombol_handler():
69      global is_muted
70
71
72      if current_state in ("WARNING", "CRITICAL", "MUTED"):
73          is_muted = not is_muted
74
75
76          status_teks = (
77              "DIHENINGKAN (MUTED)"
78              if is_muted
79              else "AKTIF KEMBALI"
80          )
81
82
83          print(
84              f"\n>>> [BUTTON DITEKAN] "
85              f"Alarm Buzzer: {status_teks} <<<"
86          )
87      else:
88          print(
89              "\n>>> [BUTTON DITEKAN] "
90              "Sistem belum dalam kondisi alarm. <<<"
91          )
92
93
94
95
96  tombol_mute.when_pressed = tekan_tombol_handler
97
98
```

```
# ============================================================
# 4. WARM-UP PIR
# ============================================================


def periksa_warmup_pir(waktu_mulai):
    global pir_siap


    if pir_siap:
        return


    waktu_berjalan = time.monotonic() - waktu_mulai
    sisa = max(
        0,
        int(WAKTU_WARMUP_PIR - waktu_berjalan),
    )


    if waktu_berjalan >= WAKTU_WARMUP_PIR:
        pir_siap = True
        print(
            "\n[PIR SIAP] Sensor gerakan siap digunakan.\n"
        )
    else:
        print(
            f"\r[PIR WARM-UP] {sisa:02d} detik tersisa",
            end="",
            flush=True,
        )




# ============================================================
# 5. PEMBACAAN SENSOR
# ============================================================


def baca_semua_sensor():
    suhu = None
    kelembaban = None


    try:
        suhu = dht_device.temperature
        kelembaban = dht_device.humidity
    except RuntimeError:
        # Jika pembacaan gagal, coba lagi pada iterasi berikutnya.
        pass
```

```
150
151
152     ada_gerakan = (
153         sensor_pir.motion_detected
154         if pir_siap
155         else False
156     )
157
158
159     return suhu, kelembaban, ada_gerakan
160
161
162
163
164 # ============================================================
165 # 6. EVALUASI STATUS
166 # ============================================================
167
168
169 def evaluasi_state(suhu, kelembaban, gerakan):
170     global current_state, is_muted
171
172
173     # Prioritas 1: kondisi kritis
174     if (
175         suhu >= BATAS_SUHU_TINGGI
176         or kelembaban >= BATAS_LEMBAB_KRITIS
177     ):
178         state_baru = "CRITICAL"
179
180
181     # Prioritas 2: kondisi peringatan
182     elif (
183         kelembaban >= BATAS_LEMBAB_WASPADA
184         or gerakan
185     ):
186         state_baru = "WARNING"
187
188
189     else:
190         state_baru = "NORMAL"
191         is_muted = False
192
193
194     # Mute tidak mengubah kondisi sensor.
195     if state_baru in ("WARNING", "CRITICAL") and is_muted:
196         current_state = "MUTED"
197     else:
198         current_state = state_baru
199
200
```

```
    return current_state



# ============================================================
# 7. KONTROL LAMPU DAN BUZZER
# ============================================================


def jalankan_output(state):
    # Hentikan pola output sebelumnya.
    buzzer.off()
    lampu.off()


    if state == "INITIALIZING":
        lampu.green.on()


    elif state == "NORMAL":
        lampu.green.on()


    elif state == "WARNING":
        lampu.yellow.on()


        buzzer.beep(
            on_time=0.2,
            off_time=0.8,
            background=True,
        )


    elif state == "CRITICAL":
        lampu.red.blink(
            on_time=0.2,
            off_time=0.2,
            background=True,
        )


        buzzer.beep(
            on_time=0.1,
            off_time=0.1,
            background=True,
        )


    elif state == "MUTED":
```

```
        lampu.yellow.on()
        lampu.red.on()
        buzzer.off()




# ============================================================
# 8. PROGRAM UTAMA
# ============================================================


def main():
    global current_state, pir_siap


    print("=== SISTEM MONITORING RUANG LAB/SERVER AKTIF ===")


    # Nilai awal sementara jika DHT11 belum berhasil dibaca.
    suhu_valid = 27.0
    lembab_valid = 60.0


    waktu_mulai = time.monotonic()
    status_output_terakhir = "INITIALIZING"


    # LED hijau sebagai indikator inisialisasi.
    jalankan_output("INITIALIZING")


    try:
        while True:
            periksa_warmup_pir(waktu_mulai)


            # DHT11 tetap dibaca selama warm-up PIR.
            suhu, kelembaban, gerakan = baca_semua_sensor()


            if suhu is not None and kelembaban is not None:
                suhu_valid = suhu
                lembab_valid = kelembaban


            # Selama warm-up, hijau tetap menyala kecuali
            # kondisi kritis suhu atau kelembapan terjadi.
            if not pir_siap:
                if (
                    suhu_valid >= BATAS_SUHU_TINGGI
```

```
                    or lembab_valid >= BATAS_LEMBAB_KRITIS
                ):
                    state_aktif = evaluasi_state(
                        suhu_valid,
                        lembab_valid,
                        False,
                    )
                else:
                    state_aktif = "INITIALIZING"
                    current_state = state_aktif
            else:
                state_aktif = evaluasi_state(
                    suhu_valid,
                    lembab_valid,
                    gerakan,
                )


            # Ubah output hanya saat status berubah.
            if state_aktif != status_output_terakhir:
                jalankan_output(state_aktif)
                status_output_terakhir = state_aktif


            if not pir_siap:
                status_pir = "WARM-UP"
            elif gerakan:
                status_pir = "GERAK"
            else:
                status_pir = "AMAN"


            print(
                f"[STATE: {state_aktif:12s}] "
                f"Suhu: {suhu_valid:.1f}C | "
                f"Lembab: {lembab_valid}% | "
                f"PIR: {status_pir:7s} | "
                f"Mute: {is_muted}"
            )


            time.sleep(2.0)


    except KeyboardInterrupt:
        print(
            "\nMenghentikan program dan membersihkan pin GPIO..."
        )


    finally:
```

```
        buzzer.off()
        lampu.off()


        tombol_mute.close()
        sensor_pir.close()


        dht_device.exit()


        buzzer.close()
        lampu.close()


        print("Semua output dimatikan dan GPIO dibersihkan.")




if __name__ == "__main__":
    main()
```

Kode 2: Kode Lengkap Sistem Monitoring Ruangan Sederhana

B. Dokumentasi Foto

Berikut adalah dokumentasi rangkaian perangkat keras dan pengujian sistem:

[Tempat Foto 1: Rangkaian Keseluruhan Sistem Monitoring Ruangan]
(Deskripsi: Rangkaian Raspberry Pi, Sensor DHT, Push Button, dan LED pada Breadboard)

[Tempat Foto 2: Pengujian Saat Kondisi Normal dan Kondisi Peringatan]
(Deskripsi: Tampilan output terminal beserta indikator LED menyala)

C. Link Video Demo Pembuatan dan Pengujian

Video demonstrasi perakitan, penjelasan alur program, serta pengujian langsung sistem dapat diakses melalui tautan berikut:

- **Tautan (URL):** https://vt.tiktok.com/ZSgaYhkMV/